



Trabajo Práctico – Unidad 02

Primeros programas: entorno, consola y entrada de datos

Cátedra: Programación en Computación — UTN FRRQ **Docentes:** Longhi Pablo, Talijancic Iván

Objetivos

Al terminar este trabajo práctico vas a poder:

-  Correr un programa con `npm run dev` sin dudar de dónde estás parado
 -  Mostrar información con `console.log()` y **template literals**
 -  Guardar datos en variables y calcular con ellos
 -  Leer datos del teclado con `prompt()` y convertirlos con `parseInt()` / `parseFloat()`
 -  Controlar cuántos decimales se muestran con `.toFixed()`
 -  Leer un mensaje de error y arreglar lo que señala
-

Consignas generales

1. **Todo se resuelve dentro del template de la cátedra.** Escribí en `src/app.js` y ejecutá con `npm run dev`.
2. **Un problema, un archivo.** Guardá cada solución aparte (`problema-1.js`, `problema-2.js`, ...) y copiala al `src/app.js` la que estés probando. Así no perdés lo anterior.
3. **Sin punto y coma** al final de las sentencias.
4. **Nombres descriptivos en inglés** para las variables: `voltage`, `current`, `dailyHours`. Nunca `x`, `a1`, `dato`.
5. Los **textos que ve el usuario van en español**, con la unidad entre corchetes: `'Tensión [V]: '`.
5. **Respetá el formato de salida** de cada ejemplo. Es lo que se compara al corregir.
7. Si algo falla, **leé el error completo** antes de cambiar el código a ciegas.

 Los valores de los ejemplos son sólo eso: ejemplos. Tu programa tiene que funcionar con **cualquier** valor que ingrese el usuario.

Problemas

Problema 1 – Tu primer programa

Escribí un programa que muestre `Hola mundo` en la consola y, en la línea siguiente, tu nombre completo.

Salida esperada:

```
Hola mundo
Iván Talijancic
```

🎯 **Objetivo real de este problema:** que el ciclo *escribir* → *guardar* → *ejecutar* te funcione. Si esto anda, el resto es cuestión de práctica.

Problema 2 – Ficha de identificación

Guardá en variables tu nombre, tu número de legajo, la carrera y el año que cursás. Después mostralos en pantalla usando **template literals** (backticks y `${}`), **no** comas.

Salida esperada:

```
— Alumno —
Nombre: Ana Gómez
Legajo: 5200
Carrera: Ingeniería Electromecánica
Año: 2.º
```

⚠ Si en la salida te aparece literalmente `${nombre}`, estás usando comilla simple `'` en lugar de backtick ```.

Problema 3 – Registro de operario

Ahora los datos no están escritos en el código: se los pedís al usuario.

Pedí por consola el **nombre** y el **número de legajo** de un operario, y mostrará un mensaje de confirmación.

Ejemplo de ejecución:

```
Nombre del operario: Ana Gómez
Número de legajo: 5200

Operario Ana Gómez (legajo 5200) registrado.
```

💡 Acordate del `import { prompt } from './prompt.js'` al principio del archivo.

Problema 4 – Conversión de temperatura

Un sensor del taller informa la temperatura en **grados Celsius**. Escribí un programa que la lea y la muestre convertida a **Fahrenheit** y a **Kelvin**, con **dos decimales**.

$$^{\circ}F = ^{\circ}C \times \frac{9}{5} + 32 \quad K = ^{\circ}C + 273,15$$

Ejemplo de ejecución:

```
Temperatura medida [°C]: 21.7
```

```
21.70 °C = 71.06 °F = 294.85 K
```

⚠ La temperatura tiene decimales: usá `parseFloat()`, no `parseInt()`. Con `parseInt`, `21.7` se convierte en `21` y perdés el decimal sin ningún aviso.

💡 Si en Kelvin te aparece `294.84999999999997`, no está mal tu cuenta: es la precisión finita que vimos en la unidad 01. Se resuelve mostrando con `.toFixed(2)`.

Problema 5 – Verificación de un circuito

Se releva una rama de un tablero midiendo tensión y corriente. Escribí un programa que lea ambos valores y calcule la **potencia** y la **resistencia equivalente**, con dos decimales.

$$P = V \times I \quad R = \frac{V}{I}$$

Ejemplo de ejecución:

```
Tensión medida [V]: 380
```

```
Corriente medida [A]: 4.2
```

```
Potencia: 1596.00 W
```

```
Resistencia: 90.48 Ω
```

Problema 6 – Ficha de consumo de un motor

Integra todo lo anterior.

El taller necesita estimar cuánto cuesta tener un motor funcionando. Escribí un programa que pida por consola:

DATO	UNIDAD	EJEMPLO
Identificación del motor	—	M-14
Tensión nominal	V	380
Corriente nominal	A	4.2
Factor de potencia (cos ϕ)	—	0.85
Horas de uso por día	h	8
Precio de la energía	\$/kWh	85.50

Y emita una ficha con:

- **Potencia aparente** $S = V \times I$ [VA]
- **Potencia activa** $P = S \times \cos \varphi$ [W]
- **Consumo diario** $E_d = \frac{P \times h}{1000}$ [kWh]
- **Consumo mensual** $E_m = E_d \times 30$ [kWh]
- **Costo mensual** $E_m \times \text{precio}$ [\$]

Todos los valores con **dos decimales**.

Ejemplo de ejecución:

```
Identificación del motor: M-14
Tensión nominal [V]: 380
Corriente nominal [A]: 4.2
Factor de potencia: 0.85
Horas de uso por día: 8
Precio de la energía [$/kWh]: 85.50
```

— Ficha de consumo —

```
Motor: M-14
Potencia aparente: 1596.00 VA
Potencia activa: 1356.60 W
Consumo diario: 10.85 kWh
Consumo mensual: 325.58 kWh
Costo mensual: $ 27837.43
```

💡 **Cómo encararlo:** primero leé los seis datos y mostralos tal cual, para verificar que los estás capturando bien. Recién después agregá los cálculos, de a uno.

💡 Guardá cada resultado intermedio en su propia variable (`apparentPower` , `activePower` , `dailyEnergy` , ...). Una sola cuenta gigante en un `console.log` es imposible de depurar.



Restricciones

- ❌ **Sin condicionales** (`if` , `switch`). Se ven en la **unidad 04**.
- ❌ **Sin bucles** (`for` , `while` , `do-while`). Se ven en la **unidad 05**.
- ❌ **Sin funciones propias**. Se ven en la **unidad 06**. Todo va escrito de corrido.
- ❌ **Sin punto y coma** al final de las sentencias.
- ✅ Sólo lo de esta unidad: variables, `console.log()` , template literals, `prompt()` , `parseInt()` , `parseFloat()` , `.toFixed()` y las operaciones `+` `-` `*` `/` .



¿Por qué tantas restricciones? Porque cada unidad se apoya en la anterior. Resolver el problema 6 sin funciones te va a resultar repetitivo — y esa incomodidad es exactamente el motivo por el que existen las funciones. En la unidad 06 vas a volver a este mismo problema y lo vas a reescribir en la mitad de líneas.



Material de consulta

- **Apunte de la unidad** (<https://github.com/italijancic/pc-2026/blob/main/unidades/02-entorno-y-primeros-pasos/apunte.pdf>)
- **Presentación de clase** (<https://github.com/italijancic/pc-2026/blob/main/unidades/02-entorno-y-primeros-pasos/presentacion.pdf>)
- **Ejemplos de la clase** (<https://github.com/italijancic/pc-2026/tree/main/unidades/02-entorno-y-primeros-pasos/ejemplos>)
- **Template del curso** (<https://github.com/italijancic/pc-2026/tree/main/template>)